

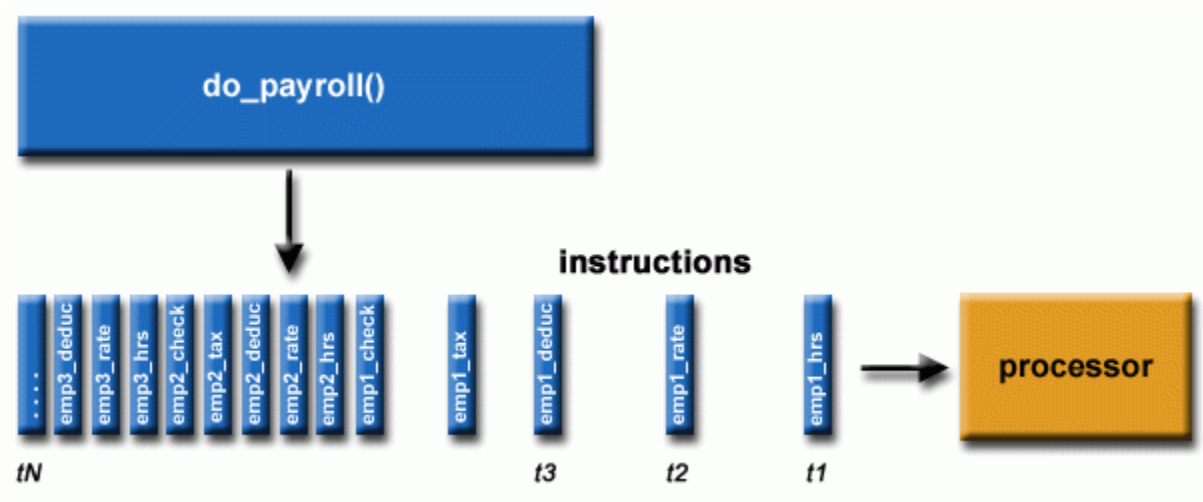
1	CO1	1. Differentiate among fundamental parallel computing models and explain their operational characteristics. 2. Analyze HPC system architectures and classify them based on suitable parallel models. 3. Apply Flynn's Taxonomy to identify and interpret architectural designs of modern supercomputers and HPC platforms. 4. Demonstrate model-to-platform mapping by evaluating real-world HPC case studies through the lens of parallel computation frameworks.	Foundations of Parallel & High Performance Computing 1.1 Motivation for Parallelism and Scope of HPC 1.2 Levels of Parallelism: Instruction, Task, Thread, Memory, Function 1.3 Parallel Models: SIMD, MIMD, SIMT, SPMD, Dataflow, Demand-Driven 1.4 HPC Architectures: Shared vs Distributed memory, Clusters, Multicores 1.5 Communication Overheads and Physical Organization of Platforms
---	-----	---	---

1.1 Motivation for Parallelism and Scope of HPC

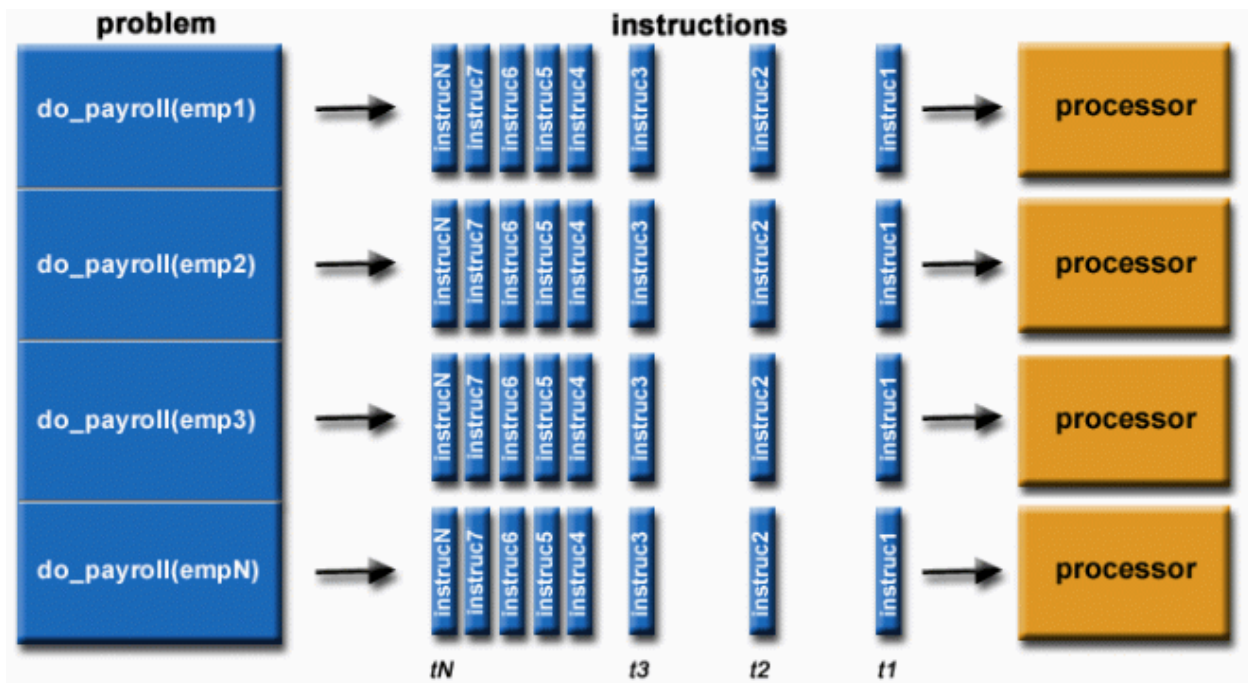
Motivation for Parallelism

- **Single core is not enough**
 - We cannot keep increasing clock speed (too much heat, power).
 - So, computers use many cores to work together.
 - Example: Instead of 1 person doing 100 math problems, 10 people can each solve 10 at the same time.
- **Large data growth**
 - Today's problems involve terabytes/petabytes of data.
 - Example: Weather forecasting needs data from thousands of sensors worldwide → parallel computing makes it possible to process all this quickly.
- **Performance demand**
 - Real-world problems (AI training, simulations) take years on one processor, but hours/days with parallelism.
 - Example: Training ChatGPT-like models is only possible using thousands of GPUs in parallel.
- **Cost efficiency**
 - Easier to connect many normal processors than design one "super" fast processor.

Serial/Normal Computing

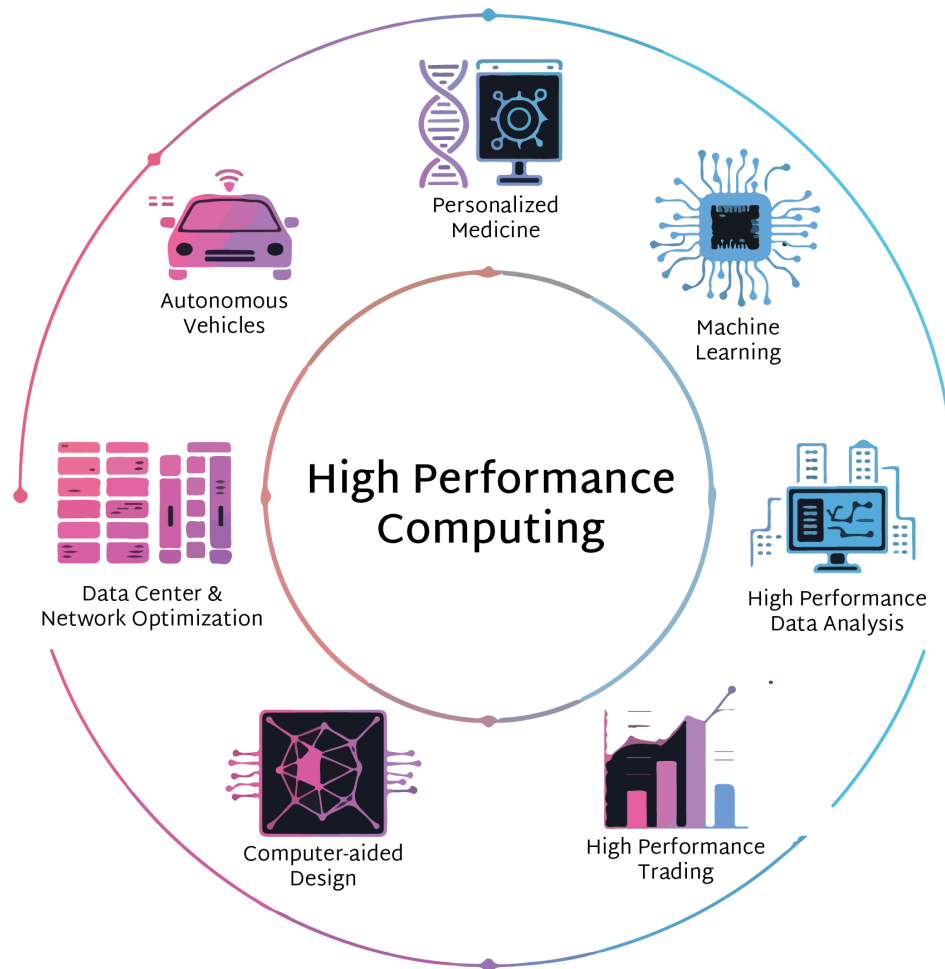


Parallel Computing



Scope of HPC

- **Definition:** HPC = using supercomputers, clusters, or parallel processing to solve large/complex problems faster.
- **Applications:**
 - Science → weather prediction, astronomy, genome sequencing.
 - Engineering → car crash simulation, aircraft design (CFD).
 - Industry/Finance → stock market modeling, fraud detection.
 - AI/ML → deep learning, image recognition at massive scale.
- **Environments:**
 - Supercomputers (e.g., Fugaku in Japan, Frontier in USA).
 - HPC clusters in universities/companies.
 - Cloud HPC (AWS, Google Cloud, Azure).

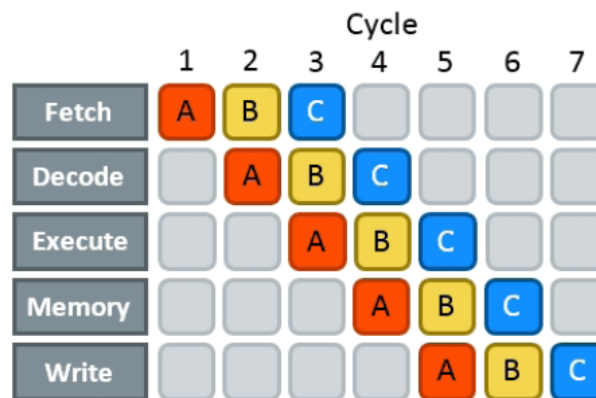


1.2 Levels of Parallelism

Parallelism can be applied at different levels in a computer system:

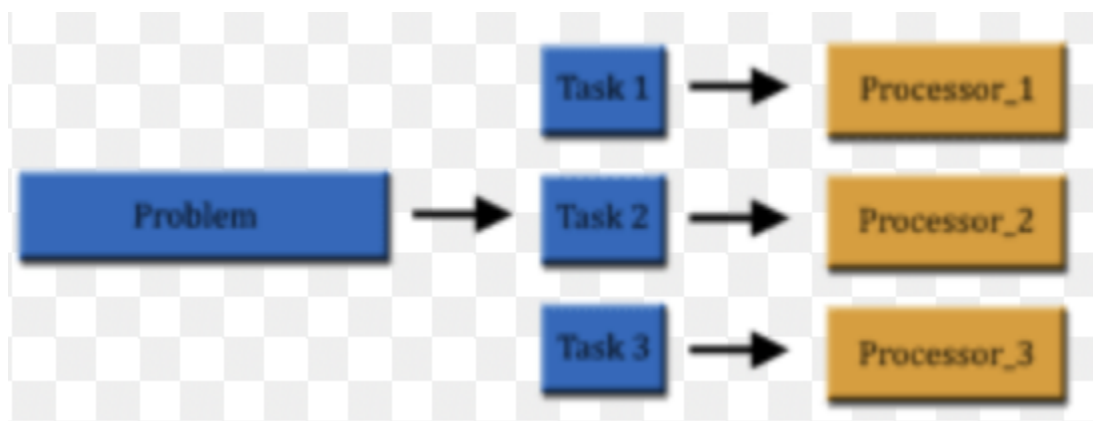
1. Instruction Level Parallelism (ILP)

- **Meaning:** A CPU executes multiple instructions at the same time.
- **How:** Uses techniques like pipelining, superscalar execution.
- **Example:**
 - While one instruction is being fetched, another is decoded, and another is executed.
 - Like an assembly line in a factory.



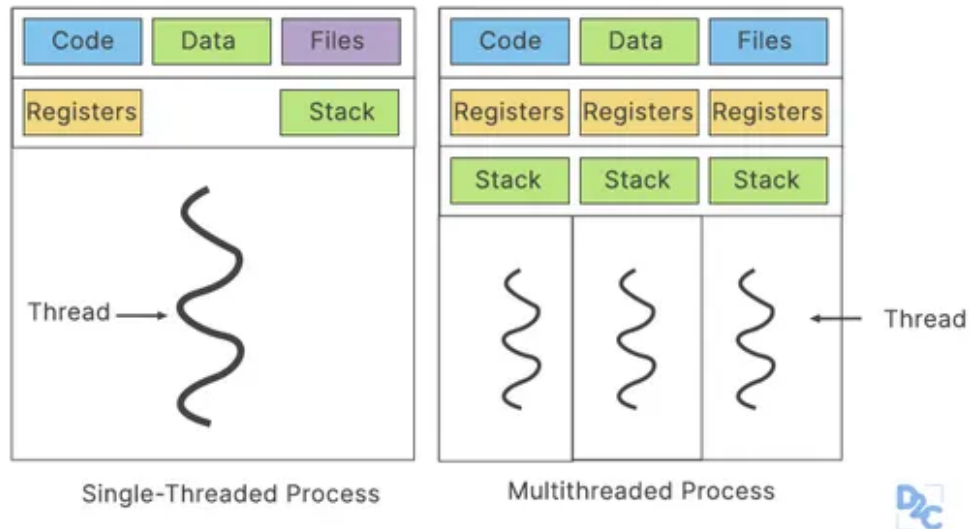
2. Task Level Parallelism

- **Meaning:** The program is divided into different tasks, each performing a different job in parallel.
- **How:** Independent tasks run on separate processors.
- **Example:**
 - In a video game → one task handles graphics, another handles sound, another handles physics.
 - All run in parallel, improving speed and responsiveness.



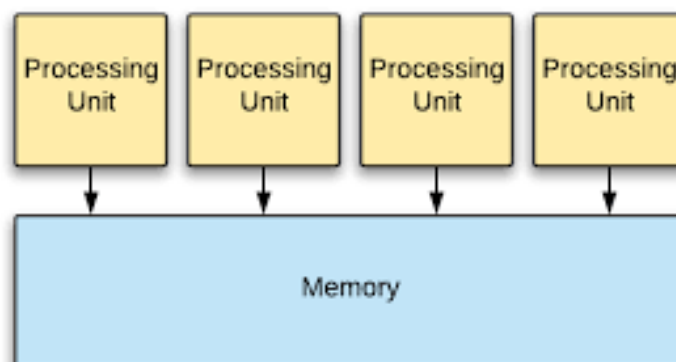
3. Thread Level Parallelism

- **Meaning:** A single program is broken into multiple threads (smaller units of execution) that can run together.
- **How:** Threads share memory but execute independently.
- **Example:**
 - A web browser → one thread loads images, another handles user input, another renders text.
 - On a multi-core CPU, these threads can run simultaneously.



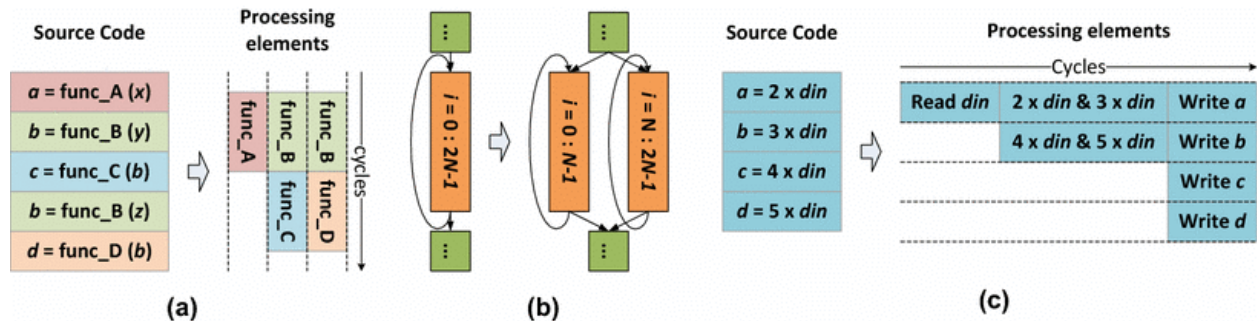
4. Memory Level Parallelism (MLP)

- **Meaning:** Hiding the delay of memory access by fetching multiple data items at once.
- **How:** Overlapping multiple memory operations (prefetching, caching).
- **Example:**
 - When adding two large arrays, the CPU fetches data for upcoming operations in advance so work doesn't stop waiting for memory.



5. Function Level Parallelism

- **Meaning:** Different functions or modules of a program run in parallel.
- **How:** Each function is executed on different cores.
- **Example:**
 - In a video streaming app:
 - One function handles video decoding,
 - Another function manages buffering,
 - Another function sends data to the display.



👉 In short:

- Instruction → Multiple instructions at once.
- Task → Independent jobs in parallel.
- Thread → Same program split into smaller parts.
- Memory → Multiple memory accesses in parallel.
- Function → Functions/modules run in parallel.

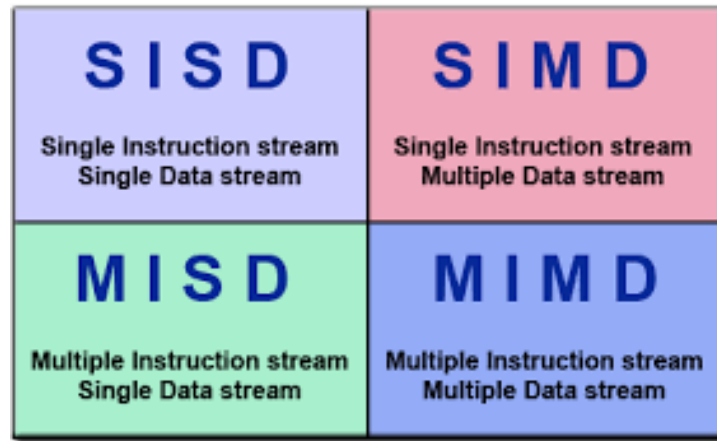
1.3 Parallel Models

Context

Early computers worked sequentially → one instruction, one data at a time.

As data grew (AI, graphics, weather), this was too slow.

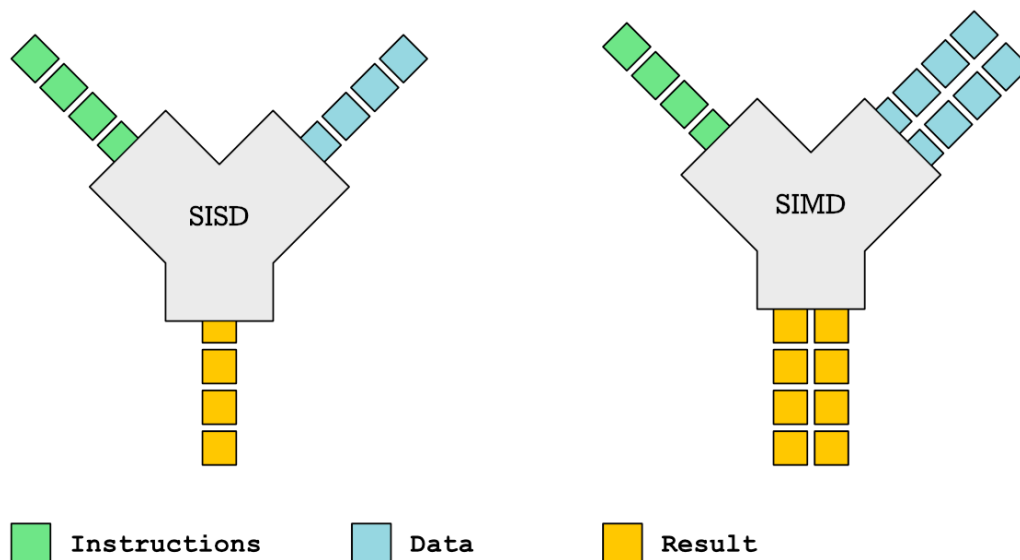
Different parallel models were created to handle different types of workloads efficiently.



Models with Examples

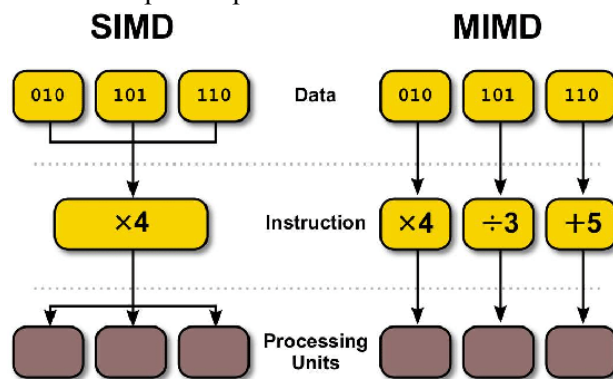
1. SIMD (Single Instruction, Multiple Data)

- **Idea:** One instruction → applied to many data items at once.
- **Why:** Best when the same math repeats on huge datasets.
- **Cool Examples:**
 - Ray tracing in graphics → millions of pixels lit up by the same math.
 - Image filtering → apply a blur/sharpen filter to all pixels at once.
 - Neural networks → matrix multiplications (core of AI) run as SIMD.
- **Impact:** Basis of GPUs and vector processors.



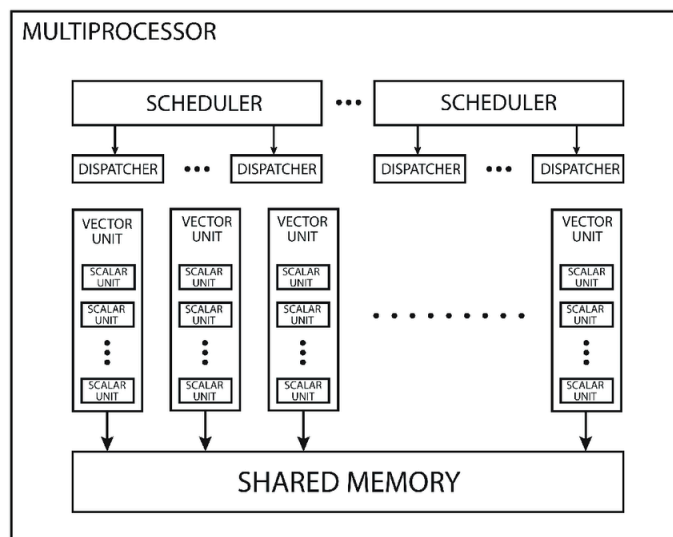
2. MIMD (Multiple Instruction, Multiple Data)

- **Idea:** Each processor can run a different instruction on different data.
- **Why:** Needed for diverse, non-uniform tasks.
- **Cool Examples:**
 - Google search engine → different servers handle crawling, indexing, ranking in parallel.
 - Large supercomputers → weather forecasting with each processor modeling a different physics equation.
 - Video streaming services → one server encodes video, another handles recommendations.
- **Impact:** Standard model in supercomputers and clusters.



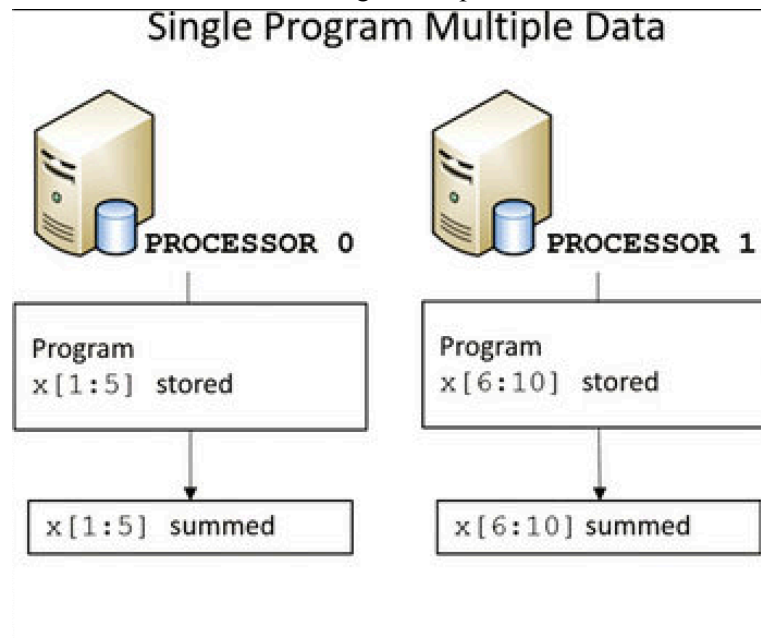
3. SIMT (Single Instruction, Multiple Threads)

- **Idea:** GPU evolution of SIMD → many threads run the same instruction, but can branch differently if needed.
- **Why:** Needed for thousands of lightweight threads.
- **Cool Examples:**
 - AI training (GPT models, image generators) → billions of operations run on thousands of CUDA threads.
 - Real-time ray tracing in games → each thread simulates light paths for different pixels.
- **Impact:** Makes GPUs the powerhouse of modern AI & graphics.



4. SPMD (Single Program, Multiple Data)

- Idea: All processors run the same program, but on different chunks of data.
- Why: Simple + scalable → fits scientific workloads.
- Cool Examples:
 - Weather forecasting → one program runs everywhere, each CPU handles a part of Earth's map.
 - Stock market risk modeling → same algorithm runs on different portfolios simultaneously.
- Impact: Common in HPC clusters using MPI/OpenMP.



5. Dataflow Model

- Idea: Execution happens when data is ready, not in strict order.
- Why: Eliminates waiting and increases throughput.
- Cool Examples:
 - Video streaming pipeline → decoding, buffering, rendering all flow as soon as data arrives.
 - Hardware accelerators (FPGAs, Tensor Processing Units) → execute operations as soon as inputs are available.
- Impact: Core idea for stream computing and accelerators.

6. Demand-Driven (Lazy Evaluation)

- Idea: Compute only when the result is required.
- Why: Saves work by avoiding unnecessary calculations.
- Cool Examples:
 - Netflix recommendations → system computes suggestions only when you log in.
 - Functional programming (Haskell) → a value is computed only if used.
 - Databases → queries optimized to compute only needed rows/columns.
- Impact: Efficient resource use in big data & programming languages.

Comparison Table

Model	Instruction	Data	Cool Example
SIMD	Single	Multiple	GPU ray tracing, image filtering, AI matrix math
MIMD	Multiple	Multiple	Supercomputers for weather, Google servers for search
SIMT	Single (per warp)	Multiple Threads	CUDA threads in GPUs for AI, gaming ray tracing
SPMD	Single Program	Multiple Data	Weather simulation (Earth regions), stock risk models
Dataflow	Data-triggered	Streams	Video pipelines, FPGAs, TPUs
Demand-Driven	On demand	Needed data only	Netflix recs, Haskell, database queries

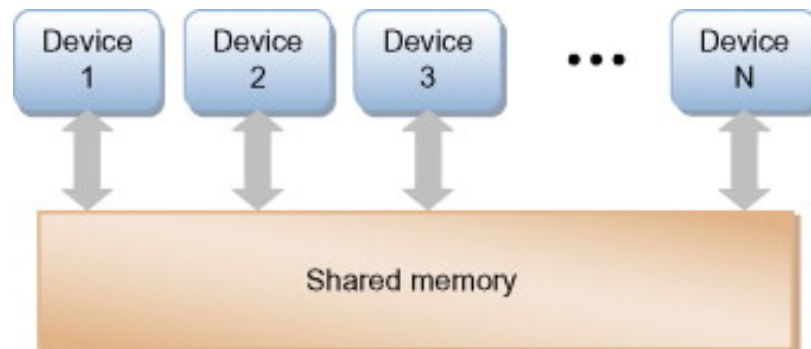
1.4 HPC Architectures

Context

To build High Performance Computers, we must decide how processors talk to memory and to each other. Different architectures evolved to balance speed, cost, and scalability.

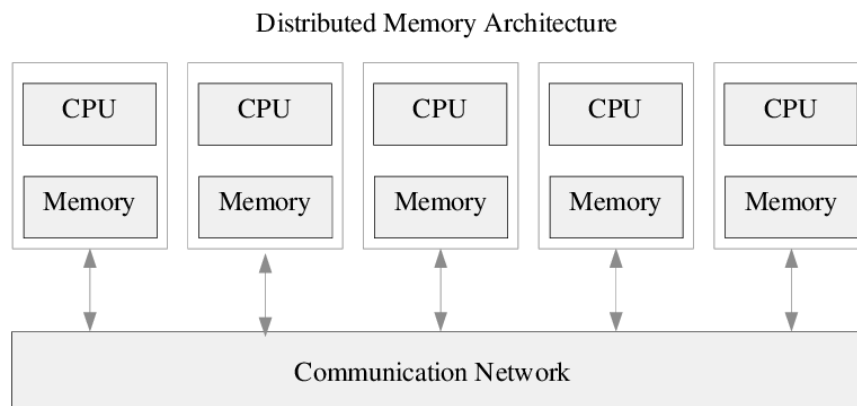
1. Shared Memory Architecture

- **Idea:** All processors share the same global memory.
- **How:** Any processor can directly access any memory location.
- **Pros:** Easy programming, fast communication.
- **Cons:** Doesn't scale well (memory bottleneck, contention).
- **Cool Examples:**
 - Your multicore laptop/desktop CPU → all cores share RAM.
 - Small-scale servers.



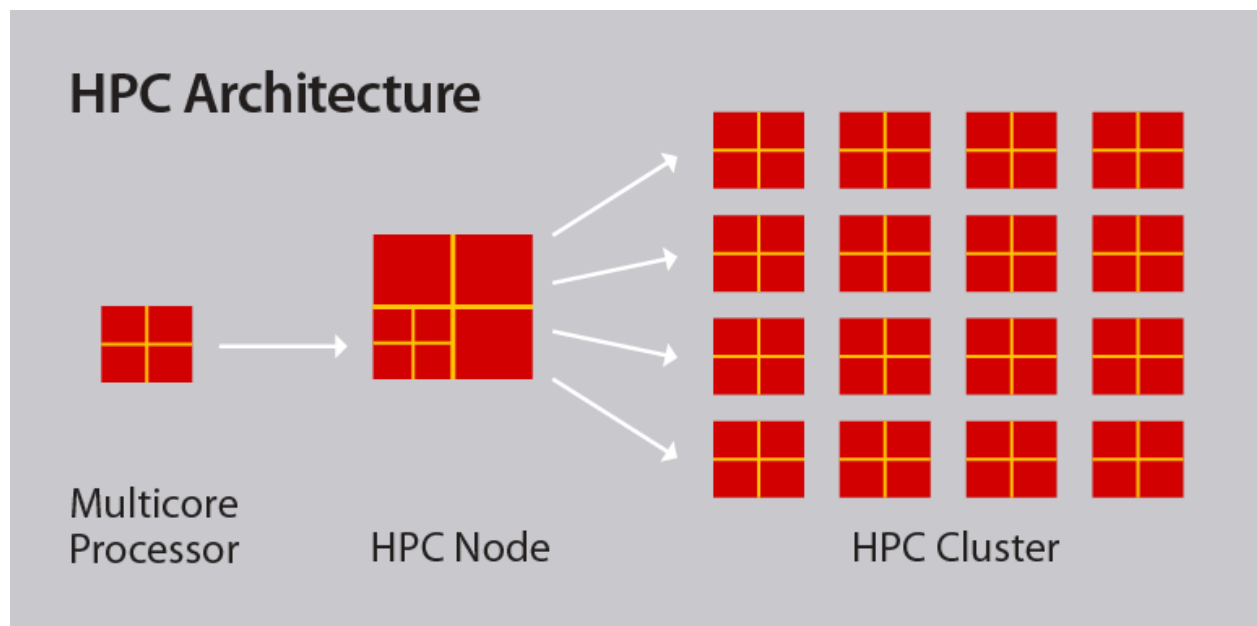
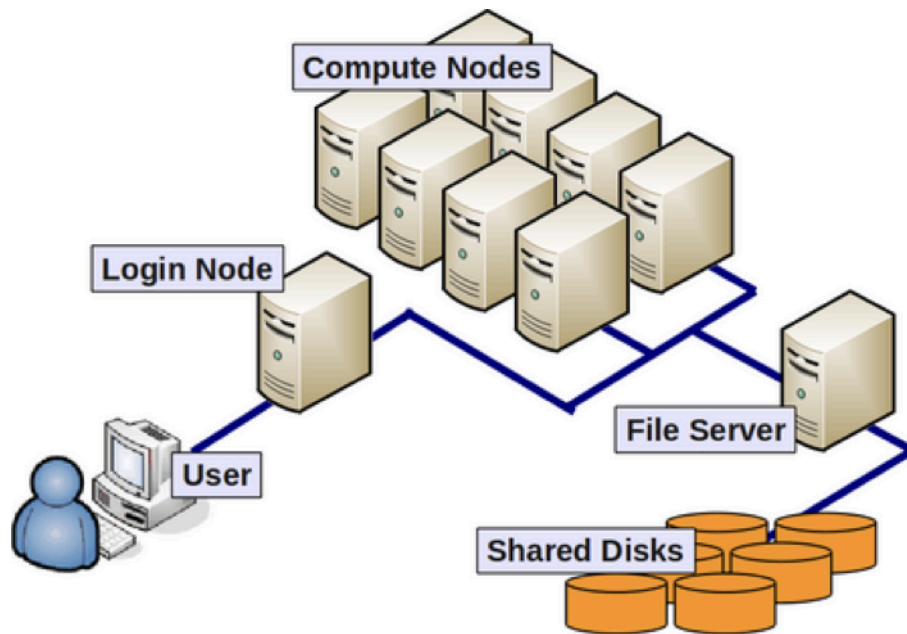
2. Distributed Memory Architecture

- **Idea:** Each processor has its own private memory.
- **How:** Processors communicate by passing messages (MPI).
- **Pros:** Scales to thousands of processors, avoids memory contention.
- **Cons:** Harder to program (must manage communication).
- **Cool Examples:**
 - Supercomputers like Frontier (USA) → each node has its own memory, connected by high-speed networks.
 - Data centers / clusters.



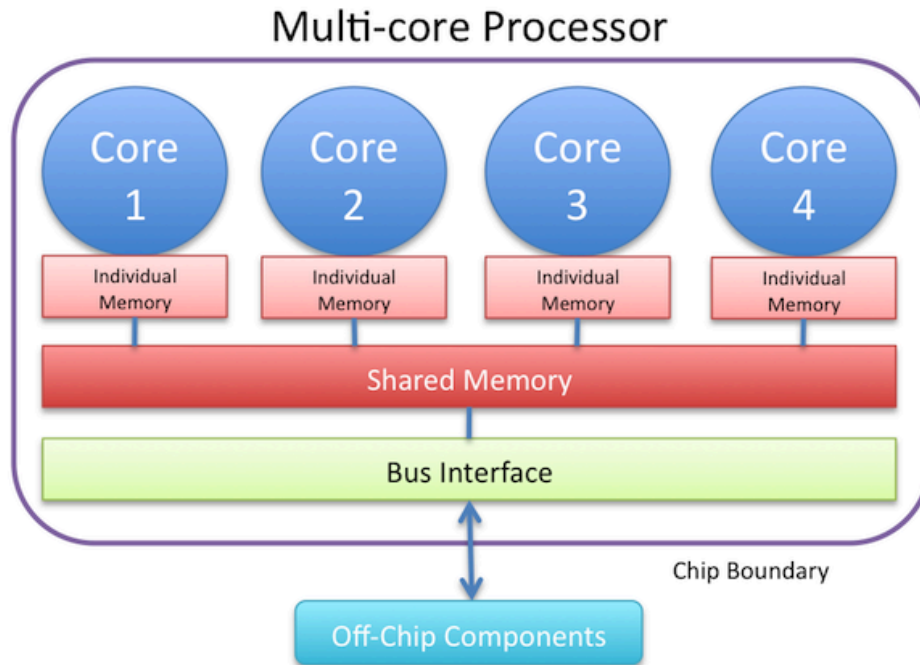
3. Clusters

- **Idea:** A group of independent computers (nodes) connected via a network to work as one.
- **How:** Each node may be shared-memory internally, but communication between nodes is distributed-memory style.
- **Pros:** Cost-effective (built from commodity hardware), scalable.
- **Cons:** Higher network latency compared to shared memory.
- **Cool Examples:**
 - Google data centers (cluster of thousands of machines).
 - Beowulf clusters in universities for research.



4. Multicores

- **Idea:** A single chip with multiple cores (mini processors) inside it.
- **How:** Each core can execute its own thread; they share caches and memory.
- **Pros:** Efficient, lower power than multiple separate CPUs.
- **Cons:** Limited by chip size and memory bandwidth.
- **Cool Examples:**
 - Intel i7, AMD Ryzen, Apple M1/M2/M3 chips → dozens of cores.
 - GPUs are “massive multicore” processors with thousands of cores.



1.5 Communication Overheads and Physical Organization of Platforms

Context

In HPC, processors often need to share data or coordinate tasks.

But communication is not free → it takes time, bandwidth, and energy.

How processors and memory are physically organized decides efficiency.

Communication Overheads

Definition: Extra cost (time + resources) when processors exchange data.

- **Latency** → Time to start a communication.
 - Example: sending a message across a cluster node might take microseconds–milliseconds.
- **Bandwidth** → Amount of data transferred per second.
 - Example: GPU-to-GPU NVLink has higher bandwidth than standard network Ethernet.
- **Synchronization cost** → Time spent waiting for others.
 - Example: In MPI, a process may wait until all others finish before proceeding.
- **Contention** → Multiple processors competing for the same resource.
 - Example: Two CPU cores trying to access the same memory bank.

👉 **Impact:** More communication overhead = less speedup from parallelism.

Physical Organization of Platforms

How processors, memory, and networks are arranged in an HPC system:

1. **Shared Memory Systems**
 - All processors connected to one memory.
 - Example: Multicore CPU in laptops/desktops.
2. **Distributed Memory Systems**
 - Each processor has its own memory; data passed via network.
 - Example: Supercomputer clusters.
3. **Hybrid Systems**
 - Nodes are shared-memory (multicore CPUs/GPUs), and nodes connect in distributed style.
 - Example: Modern exascale systems like Frontier (USA).
4. **Interconnection Networks (decide communication speed)**
 - Bus → simple, low cost, but limited scalability.
 - Crossbar → every processor has a path to every memory (fast, but expensive).
 - Multistage Networks (Omega, Butterfly) → balance speed and cost.
 - Topology-based (Mesh, Torus, Hypercube) → used in large clusters.
 - Example: IBM Blue Gene used 3D Torus; Cray supercomputers use Dragonfly topology.